FECHA: 29 DE AGOSTO DE 2026
REF: DOCUMENTACIÓN TÉCNICA

Consultorio Odontológico Dra. Silvia Pellegrino

Sitio institucional de una sola vista - Next.js 16 (App Router), 100% estático, desplegado en Vercel.

1. RESUMEN DEL PROYECTO

Sitio institucional de una sola vista (one-page) para un consultorio odontológico en La Tablada, Zona Oeste, Provincia de Buenos Aires. No hay backend, ni base de datos, ni API routes, ni formularios: el contacto se resuelve con un enlace "tel:" y un mapa embebido de Google Maps.

El repositorio tiene un único paquete, en la subcarpeta "client/". La raíz del repositorio solo contiene esa carpeta. Rama por defecto: main.

2. ARQUITECTURA

Flujo de composición y renderizado, verificado contra el código:

```
layout.tsx  (Server Component)
 |- metadata (SEO, OpenGraph, Twitter, robots, keywords)
 |- JSON-LD  <script type="application/ld+json">  -> schema.org/Dentist
 |- next/font/google  -> Instrument Serif + Playfair Display
 |- globals.scss
 `- <Preloader>              (client: overlay hasta precargar 3 imagenes)
      |- {children}
      `- <Footer>            -> Link a /Terminos
         <SpeedInsights>     (@vercel/speed-insights)

page.tsx  ("/")                            Terminos/page.tsx  ("/Terminos")
 |- <Header>          fetch /json/header.json      `- fetch /json/tyc.json
 |    `- <HeaderLinks>  react-scroll + useActiveSection    `- <LegalSection> xN
 |- <Dentista>        id=dentista
 |    |- <Presentation>      h1, bio, <a href="tel:">
 |    `- <CardsDentista>     3 tarjetas (datos hardcodeados)
 |- <Nosotros>        id=nosotros
 |    |- <LeftComponent>     foto, anima al entrar en viewport
 |    `- <RightComponent>    texto + 2 <UnorderedList>
 |- <Tratamientos>    id=tratamientos   fetch /json/tratamientos.json
 |    `- <TratamientoItem>   grilla de 8 tratamientos
 `- <Contacto>        id=contacto       fetch /json/contacto.json
      |- <ContactoMap>       <iframe> Google Maps
      `- <ContactoItem>      direccion, telefono, horarios
```

Dirección de las dependencias

- layout.tsx y page.tsx son Server Components: solo componen, no tienen lógica de navegador.
- Llevan "use client" únicamente los componentes que necesitan el navegador: Header, HeaderLinks, Preloader, Tratamientos, Contacto, LeftComponent, RightComponent y Terminos/page.tsx.
- El contenido editable (links del header, tratamientos, datos de contacto, términos y condiciones) no está en el código: vive en public/json/*.json y se pide por fetch en tiempo de ejecución.
- Excepción a esa regla: las 3 tarjetas de CardsDentista.tsx y las dos listas de RightComponent.tsx están hardcodeadas en el componente.

3. STACK TECNOLÓGICO

Versiones tal como figuran en client/package.json. Gestor de paquetes fijado: bun@1.3.14 (campo packageManager). Lockfile versionado: bun.lock.

3.1 Dependencias

Paquete	Versión	Para qué
next	16.1.2	Framework, App Router, build estático
react / react-dom	19.2.3	Librería de UI
sass	^1.101.0	Compila app/globals.scss (mixins, variables de breakpoint)
react-scroll	^1.9.3	Scroll suave del header hacia cada sección
react-intersection-observer	^10.0.3	Dispara las animaciones de entrada al llegar a la sección
@vercel/speed-insights	^1.3.1	Métricas de Core Web Vitals en Vercel
framer-motion	^12.42.2	SIN USO: no aparece importado en ningún archivo de app/

3.2 Dependencias de desarrollo

Paquete	Versión	Para qué
typescript	^5.9.3	Tipado, strict: true
eslint	^9.39.4	Linter (flat config)
eslint-config-next	16.1.2	Reglas core-web-vitals + typescript
postcss	^8.5.16	Pipeline de CSS
tailwindcss / @tailwindcss/postcss	^4.3.2	INSTALADO PERO INERTE: el plugin está en postcss.config.mjs, pero globals.scss nunca hace @import "tailwindcss" y no hay clases utilitarias en los .tsx
autoprefixer	^10.5.2	SIN USO: no está declarado como plugin en postcss.config.mjs
@types/node, @types/react, @types/react-dom, @types/react-scroll	-	Tipos

4. ESTRUCTURA DE ARCHIVOS

```
Consultorio-Odontologia/          <- raiz del repo (git)
`- client/                        <- unico paquete
   |- app/
   |  |- layout.tsx              Root layout: metadata, JSON-LD, fuentes, Preloader, Footer
   |  |- page.tsx                Home: compone las 4 secciones
   |  |- globals.scss            819 lineas. TODO el CSS del sitio
   |  |- favicon.ico
   |  |- robots.ts               Genera /robots.txt
   |  |- sitemap.ts              Genera /sitemap.xml
   |  |- lib/site.ts             Resuelve la URL canonica del sitio
   |  |- hooks/useActiveSection.ts   IntersectionObserver -> id de la seccion visible
   |  |- Terminos/
   |  |  |- page.tsx             Ruta /Terminos
   |  |  `- LegalSection.tsx     Render de una seccion legal + subsecciones
   |  `- _components/            Carpeta privada (el guion bajo la excluye del routing)
   |     |- preloader/Preloader.tsx
   |     |- header/{Header,HeaderLinks}.tsx
   |     |- dentista/{Dentista,Presentation}.tsx + cards/CardsDentista.tsx
   |     |- nosotros/Nosotros.tsx + left/LeftComponent.tsx
   |     |                        + right/{RightComponent,UnorderedList}.tsx
   |     |- tratamientos/{Tratamientos,TratamientoItem}.tsx
   |     |- contacto/{Contacto,ContactoItem,ContactoMap}.tsx
   |     `- footer/Footer.tsx
   |- public/
   |  |- json/{header,tratamientos,contacto,tyc}.json
   |  |- img/backgrounds/  img/dentist/  img/treatments/  img/us/
   |  `- design/           Flyers y mockup de diseno
   |- package.json  next.config.ts  vercel.json
   `- postcss.config.mjs  eslint.config.mjs  tsconfig.json
```

5. RUTAS

Salida verificada de "bun run build" (Next.js 16.1.2, Turbopack). Las cinco rutas son estáticas: no hay ninguna función serverless.

Ruta	Tipo	Origen
/	Static	app/page.tsx
/Terminos	Static	app/Terminos/page.tsx
/robots.txt	Static	app/robots.ts
/sitemap.xml	Static	app/sitemap.ts
/_not-found	Static	404 por defecto de Next

La navegación del header no cambia de ruta: son anclas a los id de las secciones de "/", resueltas por react-scroll.

6. MODELO DE DATOS

No hay base de datos. El "modelo" son cuatro archivos JSON servidos como assets estáticos desde public/json/, consumidos con fetch desde el cliente.

> **header.json -> LinkItem (HeaderLinks.tsx)**
> { id: number; ref: string; text: string }. El campo "ref" debe coincidir exactamente con el id del de la sección en page.tsx (dentista, nosotros, tratamientos, contacto). Si no coincide, el link no scrollea y el punto activo nunca se enciende.

> **tratamientos.json -> Tratamiento (TratamientoItem.tsx)**
> { id: number; src: string; alt: string; title: string }. "src" es una ruta absoluta bajo public/ y la imagen se renderiza con next/image a 160x160.

> **contacto.json -> Contacto (ContactoItem.tsx)**
> { id: number; subtitle: string; item: string; extra?: string }. "extra" es opcional: se usa para la segunda línea del bloque de horarios.

> **tyc.json -> LegalDataProps (LegalSection.tsx)**
> { id, title, description, subsections: [{ id, subtitle, items: string[] }] }. La "description" se omite del render si es cadena vacía. Tres secciones: Términos y Condiciones de Uso, Política de Privacidad y Contacto.

6.1 Decisiones no obvias

- Los JSON no se importan, se piden por fetch. Con resolveJsonModule: true en tsconfig.json se podrían importar y quedarían embebidos en el bundle. Se eligió fetch para poder editar el contenido sin recompilar. El costo es que la sección aparece vacía durante el primer frame y que el tipado de la respuesta es una aserción (items as LegalDataProps[]), no una validación.
- La antigüedad profesional se calcula, no se escribe: Presentation.tsx hace new Date().getFullYear() - 1981. No hay que actualizarla cada año.
- El año del footer también se calcula con new Date().getFullYear().

7. INTERFAZ Y COMPORTAMIENTO

7.1 Header flotante

HeaderLinks.tsx es la pieza con más lógica del sitio. El nav es una columna de puntos sobre el lado derecho; el texto del link está en color transparente y solo se revela en hover.

- En pantallas táctiles no hay hover: el primer toque revela el texto (clase "revealed", aplicada solo si window.matchMedia("(hover: none)") da positivo) y un pointerdown fuera del nav lo vuelve a ocultar.
- El punto azul de sección activa lo maneja useActiveSection, no el spy de react-scroll. El spy está explícitamente en false porque agregaba su propia clase "active" y quedaban dos puntos encendidos a la vez.

7.2 useActiveSection(sectionIds)

Un IntersectionObserver con rootMargin "-50% 0px -50% 0px", es decir, una franja de detección de altura cero en el centro exacto del viewport. La sección activa es la que cruza esa línea. Devuelve el id.

7.3 Preloader

Muestra un overlay a pantalla completa con un spinner hasta precargar tres imágenes: los dos fondos y us.webp. El motivo está comentado en el código: los fondos se aplican por CSS (background-image en el mixin background()) y next/image no los precarga, con lo cual aparecían de golpe. Tiene dos salvaguardas: img.onerror resuelve igual, y un failsafe quita el overlay a los 5 segundos pase lo que pase.

7.4 Animaciones de entrada

react-intersection-observer con triggerOnce: true. Al entrar en viewport se agrega la clase "inView", que dispara los keyframes fade-left / fade-right / fade-up / fade-down definidos en globals.scss. Usan animation-delay escalonado vía el mixin turn($end).

7.5 Estilos

Un único archivo, app/globals.scss, con toda la cascada anidada bajo body. Convenciones que hay que conocer antes de tocarlo:

- Los breakpoints se declaran como strings, no como media queries: $tablet: "min-width: 768px) and (max-width: 870px"; y se usan @media (#{$tablet}). Los paréntesis desbalanceados son intencionales: el de apertura lo pone el @media (y el de cierre el final de la interpolación. Es frágil, pero es la convención del archivo: si agregás un breakpoint, seguila.
- Hay 9 breakpoints en portrait ($miniMobile ... $desktop) y 4 en landscape ($mobileLandscape, $largeMobileLandscape, $tabletLandscape, $largeTabletLandscape).
- Mixins principales: titles(), subtitles(), divs() (alto y ancho de cada sección), background($url), fade($fade, $delay), turn($end) y headerLinkReveal().
- Las fuentes se consumen como custom properties que inyecta next/font: $fuente-textos: var(--font-instrument) y $fuente-titulos: var(--font-playfair).
- Los fondos usan background-attachment: fixed (efecto parallax).
- El header se posiciona con svh en vez de vh porque en móvil la barra del navegador que aparece y desaparece hacía saltar el header; se deja % como fallback en la línea anterior.
- La barra de scroll está oculta en los tres motores (scrollbar-width, -ms-overflow-style, ::-webkit-scrollbar).

8. SEO Y DATOS ESTRUCTURADOS

Todo en app/layout.tsx, app/robots.ts y app/sitemap.ts.

- metadataBase se arma con siteUrl, lo que hace que las URLs relativas de alternates.canonical, openGraph.url e imágenes se resuelvan a absolutas.
- Canonical: "/". Open Graph: type website, locale es_AR, imagen /img/us/us.webp (700x700). Twitter Card: summary_large_image.
- robots: indexable, con max-image-preview large y max-snippet -1 para Googlebot. 18 keywords declaradas en metadata.keywords.
- JSON-LD inyectado en el <body> con dangerouslySetInnerHTML, tipo schema.org/Dentist: nombre, teléfono, priceRange, medicalSpecialty, PostalAddress (Catriel 4929, La Tablada, B1766FLK, AR), GeoCoordinates y openingHoursSpecification (lunes y miércoles, 15:30 a 20:00).
- /robots.txt permite todo excepto /design, y apunta al sitemap.
- /sitemap.xml lista "/" (prioridad 1, monthly) y "/Terminos" (prioridad 0.3, yearly). lastModified se calcula en cada build con new Date().

Atención: los datos del negocio (teléfono, dirección, horarios) están duplicados entre el JSON-LD y la description de layout.tsx, y public/json/contacto.json. Si cambia un horario hay que tocar los dos.

9. SEGURIDAD

Superficie de ataque mínima: sitio estático, sin backend, sin autenticación, sin formularios y sin entrada de usuario que se persista. Lo que sí existe y conviene tener presente:

- dangerouslySetInnerHTML en layout.tsx. El contenido es un JSON.stringify de un objeto literal definido en el mismo archivo, sin datos externos: no hay vector de inyección mientras ese objeto no pase a alimentarse de una fuente externa.
- <iframe> de Google Maps (ContactoMap.tsx) con loading="lazy". Es contenido de terceros embebido; no declara sandbox ni referrerPolicy.
- Los JSON de public/ son públicos por definición. Ninguno contiene nada sensible: son textos de la página y los datos de contacto que ya están publicados.
- No hay secretos en el repositorio. El único process.env del código es VERCEL_PROJECT_PRODUCTION_URL, que es pública.
- client/.vercel/ (archivos .env.*.local bajados de Vercel y project.json con projectId/orgId) no está versionado: está en .gitignore y se confirmó con git ls-files. No commitearlo.
- /design es accesible públicamente en producción. Los flyers y el mockup están versionados y se sirven bajo https://<dominio>/design/... . robots.txt los excluye de los buscadores, pero eso no los protege: si no deben ser públicos, hay que sacarlos de public/.

10. CONFIGURACIÓN

next.config.ts
Una sola opción: allowedDevOrigins: ['localhost:3000', 'http://192.168.0.5:3000'], para poder abrir el server de desarrollo desde otro dispositivo de la red local. Esa IP es la de la máquina de desarrollo; hay que cambiarla si la red es otra.

vercel.json
framework: nextjs, installCommand: "bun install", buildCommand: "bun run build", devCommand: "bun run dev". Fuerza a Vercel a usar Bun en vez de npm. El $schema habilita autocompletado en el editor.

tsconfig.json
strict: true, target ES2017, moduleResolution bundler, alias @/* apuntando a la raíz de client/.

eslint.config.mjs
Flat config con defineConfig. Compone eslint-config-next/core-web-vitals + eslint-config-next/typescript y redeclara explícitamente los ignores por defecto (.next/**, out/**, build/**, next-env.d.ts), porque al pasar un globalIgnores propio se pisan los de la config base.

postcss.config.mjs
Único plugin: @tailwindcss/postcss. Ver la sección 12: hoy no produce ningún efecto.

11. PUESTA EN MARCHA Y DESPLIEGUE

Requisitos: Bun 1.3.14 o superior. Node 22+ si se prefiere npm, aunque el lockfile versionado es de Bun y usar npm generaría un package-lock.json que no corresponde al proyecto.

```
cd client
bun install
bun run dev        # http://localhost:3000
```

Script	Comando real	Qué hace
dev	next dev	Servidor de desarrollo con HMR
build	next build	Build de producción (Turbopack)
start	next start	Sirve el build de producción
lint	eslint	Linter sobre todo el proyecto

11.1 Despliegue

El proyecto está enlazado a Vercel (proyecto "client"; los identificadores viven en client/.vercel/project.json, que no está versionado). Vercel toma la configuración de vercel.json: instala con bun install y construye con bun run build. Root Directory del proyecto en Vercel: client. Como las cinco rutas son estáticas, el resultado es CDN puro: no se ejecuta ninguna función en cada request.

11.2 Variables de entorno

No hay .env.example ni ninguna variable que haya que configurar a mano. La única que lee el código es:

Nombre	Propósito	Obligatoria
VERCEL_PROJECT_PRODUCTION_URL	La inyecta Vercel automáticamente en build. app/lib/site.ts la usa para armar siteUrl (canonical, OpenGraph, sitemap, JSON-LD)	No: la provee la plataforma

Fuera de Vercel, siteUrl cae al valor por defecto https://silviapellegrino.com, escrito en app/lib/site.ts. Si el dominio final es otro, ese es el único lugar donde hay que cambiarlo.

12. ESTADO VERIFICADO Y PENDIENTES

Todo lo de esta sección fue comprobado contra el código, no inferido.

Build y lint en verde
"bun run lint" no reporta nada. "bun run build" compila en unos 13 segundos y prerenderiza las 5 rutas como estáticas.

No hay tests
No existe runner, ni archivos *.test.* / *.spec.*, ni script de test en package.json.

Tres dependencias no se usan
framer-motion (dependencia de producción): cero importaciones en app/; las animaciones se hacen con CSS keyframes + react-intersection-observer. tailwindcss / @tailwindcss/postcss: el plugin está activo en PostCSS, pero globals.scss nunca hace @import "tailwindcss" y ningún .tsx usa clases utilitarias, así que Tailwind no emite nada; si alguien intenta usar className="flex" no va a funcionar hasta que se agregue el import. autoprefixer: no figura entre los plugins de PostCSS.

console.log en producción
app/_components/header/Header.tsx, línea 14, imprime el JSON del header en cada carga.

client/README.md existe y está vacío
0 bytes, versionado. La documentación técnica vive en la raíz del repositorio, que es donde se abre el proyecto.

Anomalías de .gitignore
client/.gitignore se ignora a sí mismo (línea 3) e ignora /public/design (línea 5), pero ambos están versionados desde antes de que se agregaran esas reglas: .gitignore no afecta a archivos ya trackeados. Efecto práctico: los flyers de public/design/ se publican en producción.

Sin manejo de error visible
Tratamientos, Contacto y Terminos hacen fetch sin .catch(): si un JSON falla en cargar, la sección queda vacía y en silencio. Solo Header loguea el error, y a la consola, no a la interfaz.

Contenido hardcodeado que rompe la convención de los JSON
Las 3 tarjetas de CardsDentista.tsx y las listas de servicios y trayectoria de RightComponent.tsx están en el .tsx, no en public/json/.

Sin scroll-behavior: smooth en CSS
El scroll suave lo hace react-scroll en JavaScript (smooth: true, duration: 500). No hay tratamiento de prefers-reduced-motion en ninguna animación del sitio.

TODO: confirmar el dominio de producción definitivo
app/lib/site.ts usa https://silviapellegrino.com como fallback, con un comentario del propio autor advirtiendo que hay que cambiarlo si el dominio final es otro.

13. CONVENCIONES DEL REPOSITORIO

Idioma
Nombres de componentes, carpetas y clases CSS en español (Dentista, Tratamientos, nosotrosDivision); assets y sus carpetas en inglés (img/treatments, dentist.webp). Los comentarios del código están en español y explican el porqué, no el qué: mantenerlo así.

Componentes
Uno por archivo, export default, PascalCase. Carpeta por sección dentro de app/_components/, con subcarpetas cuando la sección tiene partes (nosotros/left/, nosotros/right/).

"use client"
Solo donde hace falta: hooks de navegador, fetch en useEffect o react-scroll.

Props
Interfaces declaradas en el mismo archivo del componente, con las props marcadas readonly.

Estilos
Todo en app/globals.scss. No hay CSS Modules ni styled-components. Las clases se nombran en camelCase y se anidan siguiendo la jerarquía del DOM, bajo body.

Contenido
Si el texto puede cambiar sin que cambie la lógica, va a public/json/.

Rutas de assets e imágenes
Siempre absolutas desde public/ (/img/..., /json/...). Todas las imágenes son .webp, servidas con next/image con width y height explícitos.

Commits
Los hace el titular del repositorio, no herramientas automáticas. Rama por defecto: main.

Tomás Javier Sale
Full Stack Developer